

SWIPE TO LEARN · 40 SLIDES

The Complete Guide to Caching

From "what is a cache?" to real-world patterns used by Netflix, Twitter, and Amazon — everything you need to know, one concept at a time.



Why Apps Feel Slow Without Caching

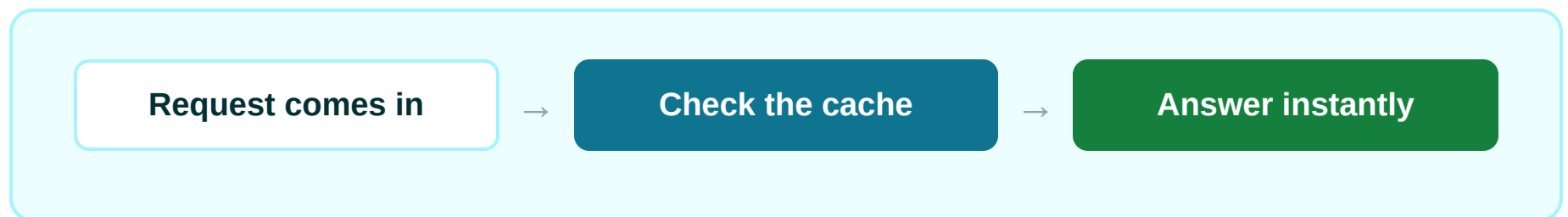
Every user request that hits your server might repeat the same expensive work — a database query, an API call, a file read — even if the answer hasn't changed since the last request.

- **A news homepage** runs the same "top 10 articles" query thousands of times per minute — even though editors publish maybe once an hour.
- **A product page** re-fetches the same price and description from the database on every single page view.
- **A dashboard** re-runs heavy aggregation queries on each refresh, even though the underlying data hasn't changed.

The result: wasted compute, higher infrastructure costs, and a slow user experience the moment real traffic arrives. **Caching solves all three.**

What Is Caching?

Caching is the practice of **storing a copy of data in a faster location** so future requests for that same data can be served instantly instead of being recomputed or re-fetched from the slow original source.



- **Simple analogy:** a café barista remembers your order. Instead of asking every morning, they start making it when you walk in. The result is stored in memory — the cache.
- **What gets cached:** database query results, API responses, rendered HTML, computed values, images, session tokens.
- **Key rule:** only cache data that is *read more often than it changes*.

Cache Hit vs Cache Miss

1

Cache Hit — the fast path

The requested data is found in the cache and returned immediately. No database call, no recomputation. Response time drops from hundreds of milliseconds to under 2ms.

2

Cache Miss — the slow path

The data isn't in the cache (either never cached or expired). The system fetches it from the original source, stores the result in cache, then returns it to the user.

3

Cache Hit Rate

The percentage of requests served from cache. A hit rate of 90% means only 1 in 10 requests ever touches the database. Most production systems target 85–99%.

4

Why the first miss is worth it

One slow request to populate the cache saves every subsequent request from being slow. Paying once to benefit thousands of times is the whole point.

Why Speed Difference Is Enormous

The performance gap between a cache and the original data source is not 10% or 2x — it's often **100x to 10,000x**.

CPU cache (L1)	→	~1 nanosecond
RAM / in-memory (Redis)	→	~100 microseconds
SSD disk read	→	~100 milliseconds
Remote database query	→	500ms – 2,000ms

At 10,000 requests per second, a 200ms database query means your system spends **2,000 CPU-seconds per second** just on data fetching. A Redis hit costs nothing by comparison.

Where Caching Lives in a System

CPU**Hardware Cache (L1/L2/L3)**

Built into the processor chip — caches recently used CPU instructions and data. Fully automatic, no code required.

BR**Browser Cache**

Stores HTML, CSS, JS, fonts, and images on the user's device so repeat page visits don't re-download the same assets.

CDN**CDN Cache**

Copies of your static and sometimes dynamic content stored in edge servers around the world, close to end users.

APP**Application Cache**

In-memory stores like Redis or Memcached sitting between your app server and your database.

DB**Database Cache**

Query result caches and buffer pools built into the database engine itself — MySQL, PostgreSQL, and others all do this internally.

Browser Caching

Explained

When a browser downloads a file — a stylesheet, a JS bundle, a logo — it can store that file locally on the device and reuse it on the next visit **without any network request at all**.

- **How it's controlled:** HTTP response headers like *Cache-Control*, *Expires*, and *ETag* tell the browser exactly how long to keep a cached copy and when to check for updates.
- **Example:** *Cache-Control: max-age=31536000* tells the browser it can reuse this file for a full year before checking again.
- **Hard refresh (Ctrl+Shift+R):** forces the browser to ignore its cache and re-download everything — useful for debugging but invisible to end users.
- **Cache busting:** adding a version hash to filenames (e.g. *app.a3f2c9.js*) ensures users pick up new versions when you deploy, since the filename itself changed.

CDN Caching

Explained

A CDN (Content Delivery Network) is a global network of servers that stores copies of your content close to users. Instead of every request travelling to your origin server, it's answered by the nearest **edge node**.

- **Example:** your origin server is in Mumbai. A user in London requesting your homepage hits a CDN edge node in London instead — latency drops from ~200ms to ~10ms.
- **What CDNs cache:** originally just images and videos, but modern CDNs like Cloudflare and Fastly also cache dynamic API responses and entire HTML pages.
- **Cache invalidation on CDN:** when you update content, you send a "purge" request to the CDN to delete the old cached copy from all edge nodes globally.
- **Why big companies care:** Netflix caches the most popular shows on thousands of ISP-level edge servers — so your movie is already physically nearby before you press play.

Application-Level Caching Explained

Application caching means your backend code explicitly stores computed results — query output, API responses, session data — in a fast in-memory store, and reads from it before touching the database.

What gets stored

- Database query results
- Rendered page fragments
- Third-party API responses
- User session tokens
- Feature flag values

Tools used

- Redis — most popular, feature-rich
- Memcached — simple, ultra-fast
- Hazelcast — distributed Java cache
- In-process dictionaries (for single servers)
- Ehcache, Caffeine (JVM)

Example: instead of querying 3 tables to build a user's dashboard every time, cache the final result object in Redis with a key like *dashboard:user:4521* for 5 minutes.

Database-Level Caching Explained

Databases run their own internal caches automatically — you don't write any code for this; the database engine handles it silently in the background.

- **Buffer Pool (InnoDB / PostgreSQL):** the database keeps recently accessed table pages in RAM. If the same rows are read again, they come from memory, not disk.
- **Query Cache (MySQL, deprecated):** stored exact query strings and their result sets. Removed in MySQL 8 because cache invalidation was too expensive at scale.
- **Execution plan cache:** databases cache the optimized plan for a query so they don't re-plan on every identical call — huge savings for frequently run queries.
- **Read replicas as cache:** sending read queries to a replica reduces load on the primary write database — a form of horizontal caching at the infrastructure level.
- **Materialized views:** pre-computed query results stored as real tables — refreshed on a schedule, not on every query.

CPU Cache:

The Fastest Cache Alive

Before RAM, before Redis, before any software cache — your CPU has its own tiny, blindingly fast memory built right onto the chip called the **CPU cache**.

L1 Cache

32–64 KB. Sits inside each processor core.
Access time: ~1 nanosecond. Stores the instructions and data being used right now.

L2 Cache

256 KB – 1 MB per core. Slightly slower (~5ns) but larger. Acts as a buffer between L1 and the shared L3.

L3 Cache

8–64 MB, shared across all cores. ~10–20ns.
Bridges the gap between the blazing-fast L2 and much-slower main RAM.

Why it matters for devs

Writing data structures that fit in cache lines (64 bytes) — "cache-friendly code" — can make algorithms 10x faster without changing the algorithm itself.

DNS Caching:

The One You Never Think About

Every time you visit a website, your device needs to translate www.google.com into an IP address like [142.250.67.14](https://www.google.com). This lookup is called a DNS query — and it's cached at multiple levels so you don't pay for it every time.

- **Your OS cache:** stores recent DNS answers in memory. Most lookups never leave your machine.
- **Router cache:** your home router caches DNS results for everyone on the network.
- **ISP resolver:** your internet provider's DNS servers cache popular domains shared across millions of customers.
- **TTL controls freshness:** each DNS record has a TTL (e.g. 300 seconds). After it expires, the next request goes all the way back to the authoritative DNS server.
- **Deployment trick:** before a domain migration, lower your TTL to 60 seconds a day in advance — so the old IP stops being cached quickly when you cut over.

Cache Hit Rate:

How You Measure Success

Cache Hit Rate = (Cache Hits ÷ Total Requests) × 100

If 9,500 out of 10,000 requests are served from cache, your hit rate is **95%**. That means only 500 requests actually cost you a database call.

What's a good hit rate?

- Below 70% → cache may be poorly keyed or TTL too short
- 70–85% → acceptable for changing data
- 85–99% → strong — most workloads aim here
- 99%+ → exceptional, typical for static assets

What lowers hit rate?

- TTL set too short — items expire before reuse
- Cache keys too specific — too many unique keys
- Highly personalised content — every user has unique data
- Cold start — a fresh cache after a restart

TTL — Time To Live

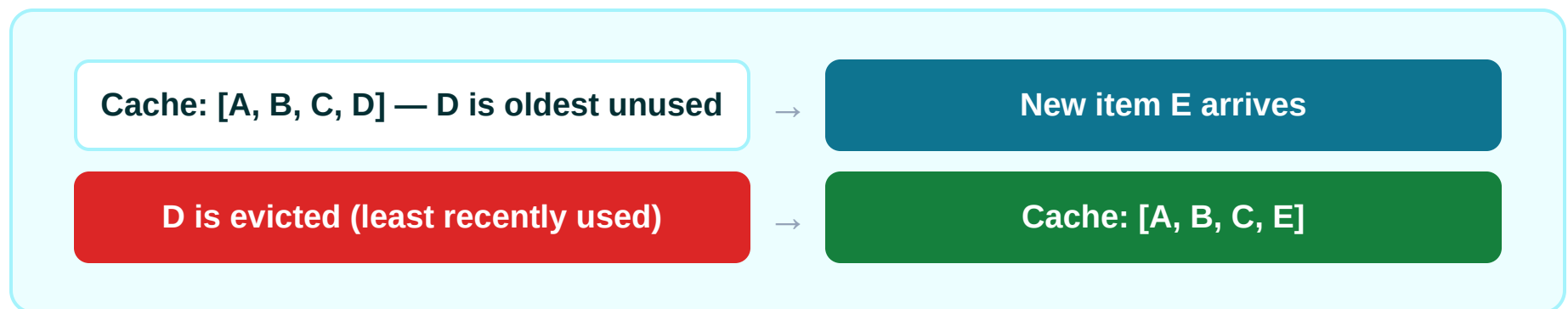
Explained

TTL is the lifespan of a cached item — the number of seconds it stays valid before it is automatically deleted and must be re-fetched on the next request.

- **Short TTL (5–60 seconds):** great for fast-changing data like live sports scores, stock prices, or real-time inventory. Keeps data fresh but results in more cache misses.
- **Medium TTL (1–60 minutes):** suits data that updates periodically — news feeds, product listings, user profiles. Balances freshness with performance.
- **Long TTL (hours to days):** perfect for slow-changing or static data — configuration settings, navigation menus, holiday schedules.
- **No TTL (no expiry):** only safe for data that truly never changes, or where you manage invalidation manually.
- **Example in code:** `redis.setex("top_articles", 300, json_data)` — stores the data and auto-deletes after 300 seconds (5 minutes).

LRU: Least Recently Used

When a cache is full and a new item needs space, LRU removes the item that was **accessed least recently** — the assumption being that if you haven't needed it in a while, you probably won't need it soon.



- **Best for:** general-purpose caches where recent usage strongly predicts future usage.
- **Used by:** Redis (when maxmemory-policy is set to allkeys-lru), Linux page cache, most browser caches.
- **Weakness:** a large one-time scan of many unique items "pollutes" the cache, evicting hot items that were genuinely popular.

LFU: Least Frequently Used

LFU evicts the item that has been **accessed the fewest total times** — not just the least recently, but the least overall. It favors data that has proven popular through many repeated accesses.

- **Example:** item A was accessed 200 times last week and 0 times today. Item B was accessed just 3 times ever, but 2 of those were today. LRU would keep B; LFU might keep A if its historical count is higher.
- **Best for:** content popularity caches — social media trending posts, top product recommendations — where truly popular items should stay regardless of a recent quiet period.
- **Weakness:** older items can accumulate very high counts from the past and "stick" in cache even if they've become stale or irrelevant now. Some implementations add a time-decay factor to handle this.
- **Used by:** Redis 4.0+ supports LFU as an eviction policy option.

FIFO & Random: Simpler Evictions

Not every cache needs complex access-tracking. Simpler policies work well when the data doesn't have strong popularity patterns.

1

FIFO — First In, First Out

Evicts the item that entered the cache first, regardless of how often or how recently it was used. Predictable, easy to implement, but ignores actual usage patterns entirely.

2

Random Replacement

Picks a random item to evict when space is needed. Surprisingly performs close to LRU in many workloads, with much lower implementation overhead.

3

MRU — Most Recently Used

The opposite of LRU: evicts the most recently accessed item. Sounds counterintuitive but works well when you know that recently used items are unlikely to be reused soon (e.g. media streaming buffers).

Cache Invalidation: The Hardest Problem

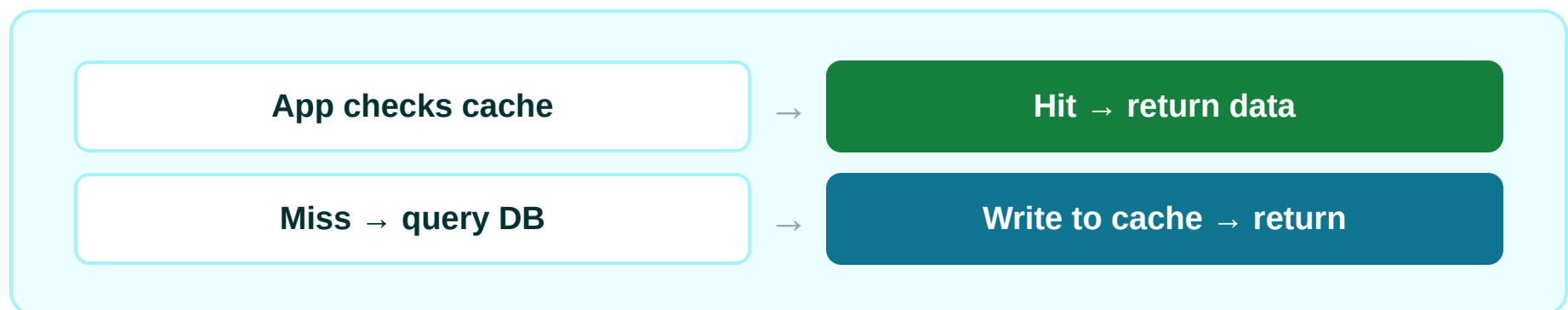
Phil Karlton's famous quote: *"There are only two hard things in Computer Science: cache invalidation and naming things."*

Once you cache something, you have **two copies of truth**. The moment the original data changes, your cache is wrong — and serving wrong data is often worse than serving slow data.

- **Example:** a user changes their email address. Your app updates the database — but the user object is still cached for 10 more minutes. Other parts of the system send emails to the old address.
- **Strategy 1 — Short TTL:** set the TTL low enough that stale data expires quickly. Simple, but you still have a stale window.
- **Strategy 2 — Active invalidation:** whenever the source data changes, explicitly delete (or update) the cache key. Eliminates the stale window but adds complexity.
- **Strategy 3 — Event-driven purge:** use events (e.g. database triggers, message queues) to automatically invalidate downstream caches when source data changes.

Cache-Aside (Lazy Loading)

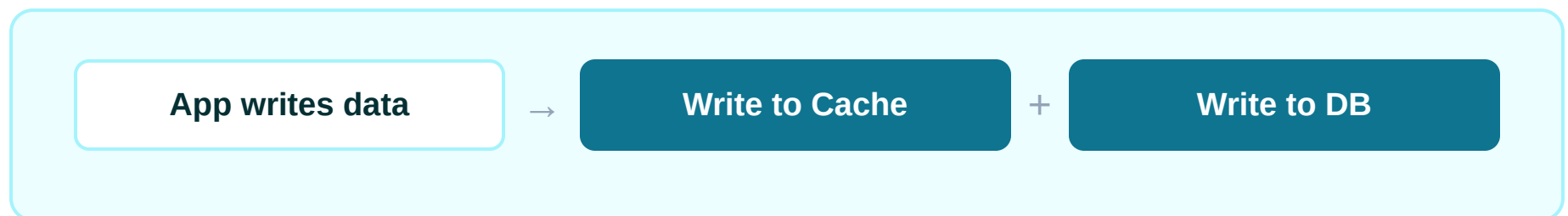
The most common caching pattern. The application code owns all cache reads and writes — the cache is never populated upfront, only when data is first requested.



- **Advantage:** only caches data that is actually requested — no wasted memory on data nobody asks for.
- **Advantage:** the cache can go down without breaking the app — it just becomes slower, not broken.
- **Disadvantage:** first request always misses. On cold start or after expiry, a burst of requests can all miss simultaneously.
- **Used by:** most web apps — Django, Rails, and Laravel all use this pattern in their default cache integrations.

Write-Through Caching

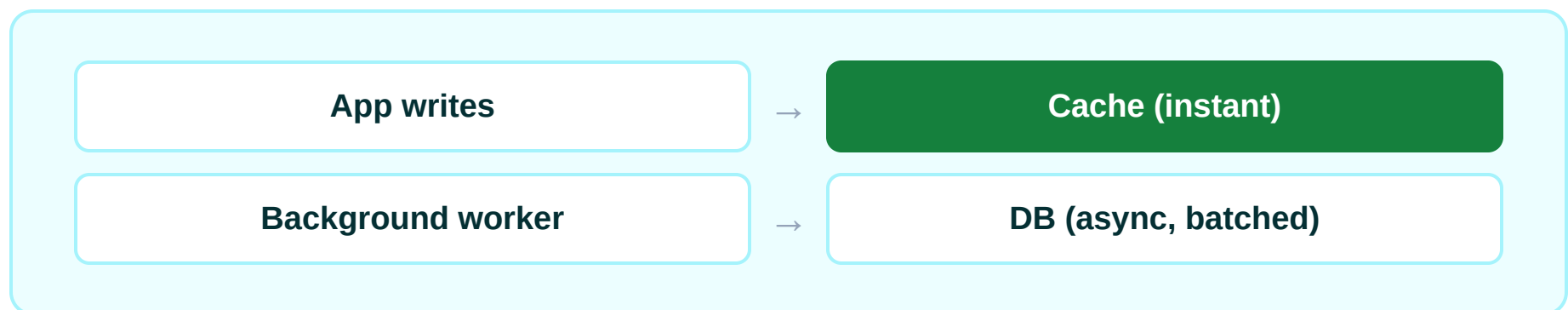
Every write goes to **both the cache and the database simultaneously**. The cache is always up-to-date because it is updated at the exact same moment as the source.



- **Advantage:** cache is never stale. Every read will get the latest version because the write always kept them in sync.
- **Advantage:** reads are always fast — no cold start problem, since data is in cache from the moment it's first written.
- **Disadvantage:** writes are slightly slower — you wait for both the cache and the database to confirm the write before returning.
- **Disadvantage:** wastes cache space on data that is written frequently but rarely read.
- **Best for:** systems where read consistency is critical — banking balances, order status, inventory counts.

Write-Back (Write-Behind)

Writes go **to the cache only** first. A background process asynchronously flushes the cached changes to the database later, in batches.



- **Advantage:** writes feel almost instant to the user — no waiting for a slow database confirmation.
- **Advantage:** multiple rapid writes to the same key can be collapsed into a single database write — great for things like view counts or like counters.
- **Disadvantage:** if the cache server crashes before the background flush, those writes are permanently lost — data loss risk.
- **Best for:** high-write, low-read scenarios — analytics event counters, real-time game scores, social media like/view tallies.

Read-Through Caching

Similar to cache-aside, but the **cache layer itself** handles the miss — not the application code. The app only ever talks to the cache; the cache internally fetches from the database when needed.

- **How it differs from Cache-Aside:** in cache-aside, the app manually checks the cache, fetches from DB on miss, then writes to cache. In read-through, the app only calls `cache.get(key)` — the cache library does the rest automatically.
- **Advantage:** application code is cleaner — no manual "check cache → miss → query DB → write to cache" logic scattered throughout the codebase.
- **Advantage:** cache population logic lives in one place — the cache configuration — not spread across many service methods.
- **Disadvantage:** more complex cache layer to set up. Requires a cache library or proxy that supports this natively (e.g. Ehcache, AWS ElastiCache with proper config).
- **Best for:** read-heavy systems where you want to simplify application logic.

Cache Stampede: When Everyone Misses at Once

A cache stampede (also called "thundering herd") happens when a popular cached item expires and **thousands of requests arrive simultaneously** — all of them getting a cache miss, all running the expensive query at the same time.

Popular key expires at 12:00:00

→

5,000 misses hit DB in 1 second

- **Real impact:** this is how caching can ironically *cause* a database outage — the very system designed to protect the DB instead causes a sudden spike that takes it down.
- **Solution 1 — Mutex Lock:** when a miss is detected, one request acquires a lock, fetches from DB, and populates the cache. All other requests wait briefly and then read from the freshly populated cache.
- **Solution 2 — Probabilistic early expiry:** slightly before TTL expires, start re-computing the value so the cache is refreshed before it ever goes stale.
- **Solution 3 — Jitter on TTL:** instead of all keys expiring at exactly the same time, add a random offset (e.g. $TTL \pm 10\%$) so expirations are spread out.

Cache Penetration: Flooding With Non-Existent Keys

Cache penetration happens when requests are made for data that **doesn't exist anywhere** — not in cache, not in the database. Every request misses the cache and hits the database for a result that is always empty.

- **Example:** an attacker sends requests for user IDs 999999999, 888888888, 777777777 — IDs that don't exist. The cache always misses. The database query runs every time. The server is hammered.
- **Solution 1 — Cache null responses:** when a DB query returns nothing, cache that "nothing" result with a short TTL (e.g. 30 seconds). Future requests for that key hit the cache and return empty instantly.
- **Solution 2 — Bloom filter:** a probabilistic data structure that sits in front of the cache. It can definitively say "this key definitely does NOT exist" — blocking the request before it ever reaches the cache or DB.
- **Solution 3 — Input validation:** validate request parameters before they even reach the cache layer — reject obviously invalid IDs early.

Cache Avalanche: Mass Expiry at Once

Cache avalanche is when a **large number of cache keys expire at the same time**, causing a sudden flood of database queries simultaneously — like a dam breaking all at once.

- **Common cause:** after a server restart, all cache keys are gone simultaneously. Or when batch loading sets all keys with the same TTL and they all expire together.
- **Example:** an e-commerce site caches every product page with a 1-hour TTL, all set at midnight. At 1:00 AM, all 50,000 product pages expire at exactly the same time — and the database gets 50,000 simultaneous queries.
- **Solution 1 — TTL jitter:** add a random spread to TTLs (e.g. $3600 + \text{random}(0-600)$ seconds) so expiry happens across a 10-minute window, not at one instant.
- **Solution 2 — Cache warming:** before a deployment or restart, pre-populate the cache with the most important keys so it's never truly cold.
- **Solution 3 — Circuit breaker:** temporarily serve stale data rather than hammer the database when a sudden miss spike is detected.

Cache Keys:

The Foundation of Good Caching

A cache key is the unique identifier used to store and retrieve a cached value. **Poor key design** is behind many of the most common caching bugs.

Good key design

- Hierarchical and readable: *user:4521:profile*
- Includes all parameters that affect the result: *products:cat:shoes:page:3*
- Namespaced to avoid collisions between services
- Version-tagged when schema changes: *v2:user:4521*

Bad key design

- ✗ Too broad: *"products"* — returns wrong results for different filters
- ✗ Too narrow: includes timestamp down to milliseconds — every request is unique, zero reuse
- ✗ No namespace — collisions between modules sharing the same cache store
- ✗ Sensitive data in the key — user passwords, PII in plaintext

Pre-Loading the Cache Before Traffic Arrives

Cache warming (also called "priming") means **proactively populating the cache** with known-important data before users start requesting it — so the cache is warm, not cold, when traffic hits.

- **When it's critical:** after a server restart (cache lost), a new deployment (fresh cache), or a planned traffic spike like a product launch or a flash sale.
- **Example — e-commerce:** before Black Friday, run a background job that visits every top-1000 product page, populates the cache, and confirms all keys are warm before the sale begins.
- **Example — media:** before a big movie release at midnight, Netflix pre-pushes the video files to thousands of edge ISP servers so customers hit local caches, not the origin, at the moment of release.
- **Predictive warming:** using access logs to identify which keys will be needed, then warming those specific keys in advance based on historical patterns.
- **Lazy warming alternative:** route a small percentage of real traffic to the new server before full switch-over, letting real usage warm the cache organically.

Redis: The World's Most Popular Cache

Redis (Remote Dictionary Server) is an open-source, in-memory data store used as a cache, session store, pub/sub broker, and message queue by millions of applications worldwide.

Core data structures

- Strings — the basic key-value cache
- Hashes — store full objects (like user profiles)
- Lists — queues and timelines
- Sets & Sorted Sets — leaderboards, tags
- Bitmaps — feature flags, presence tracking

Key features for caching

- Atomic TTL — auto-expire any key
- Persistence — can optionally save to disk
- Pub/Sub — invalidate caches via events
- Lua scripting — complex atomic operations
- Cluster mode — distributes data across nodes

Who uses it: Twitter, GitHub, Snapchat, StackOverflow, Airbnb, and virtually every high-traffic web application in production today.

Redis vs Memcached

Both are in-memory key-value stores used for caching. Memcached came first (2003) and is simpler. Redis (2009) added richer features that made it the dominant choice today.

Feature	Redis	Memcached
Data structures	Strings, Hashes, Lists, Sets, etc.	Strings only
Persistence	Yes (RDB + AOF)	No — memory only
Replication	Built-in master-replica	Not built-in
Pub/Sub	Yes	No
Lua scripting	Yes	No
Best for	Most use cases today	Simple string caching at high scale

HTTP Caching Headers

You Must Know

Cache-Control: max-age=3600

Tells the browser (and any intermediate CDN) to cache this response and treat it as fresh for 3600 seconds (1 hour) without making another request.

Cache-Control: no-cache

Misleadingly named — doesn't disable caching. It means: cache this, but check with the server on every request before using the cached copy.

Cache-Control: no-store

Never cache this response anywhere — not in browser, not in CDN. Used for sensitive data like bank statements or one-time codes.

ETag: "abc123"

A fingerprint (hash) of the response. The browser sends it back on subsequent requests; if the server's ETag matches, it replies 304 Not Modified — saving bandwidth since no body is sent.

Vary: Accept-Language

Tells the CDN to cache separate versions of a response for each different language header — so French and English users get different cached responses for the same URL.

Distributed Caching:

Caching Across Many Servers

A single cache server becomes a bottleneck and a single point of failure at scale.

Distributed caching spreads cache data across **multiple nodes**, so no single machine holds everything.

- **Consistent hashing:** the technique used to decide which cache node holds which keys. When a node is added or removed, only a fraction of keys need to be remapped — not all of them.
- **Example:** a cluster of 5 Redis nodes. Key *user:4521:profile* always maps to node 3 based on a hash of the key. Any server in your fleet asking for that key goes directly to node 3.
- **Replication:** each key can be stored on 2 or more nodes simultaneously. If node 3 goes down, node 4 has a replica and serves the data without a miss.
- **Redis Cluster:** built-in distributed mode that automatically shards data across 16,384 hash slots across up to 1,000 nodes.
- **AWS ElastiCache / GCP Memorystore:** managed distributed cache services that handle node management, failover, and replication automatically.

When Caching Makes Things Worse

Caching is powerful, but caching the wrong data creates bugs that are **very hard to debug** because the symptom (wrong data) looks completely unrelated to the cause (stale cache).

Never cache these

- ✗ Passwords, authentication tokens
- ✗ Financial transaction results
- ✗ Real-time stock prices (under seconds TTL)
- ✗ One-time codes (OTP, reset links)
- ✗ Per-user, real-time data (e.g. live cart)

Great things to cache

- ✓ Product catalogue listings
- ✓ Top-N leaderboards and rankings
- ✓ Public user profiles
- ✓ Static configuration values
- ✓ Third-party API responses

Rule of thumb: if wrong data would cause a financial, security, or safety issue — don't cache it, or cache with extreme caution and very short TTLs.

How to Measure Cache Performance

1

Hit Rate %

The primary KPI. $(\text{Cache Hits} \div \text{Total Requests}) \times 100$. Target above 85% for most application caches, 99%+ for static assets.

2

Latency Percentiles (P50, P95, P99)

Track the median and tail latency for both cache hits and misses. A high P99 miss latency reveals slow fallback queries that need optimization.

3

Memory Usage & Eviction Rate

If the eviction rate is high, the cache is too small for its workload and is constantly throwing out data before it can be reused.

4

DB Query Rate

The most practical signal: if caching is working, the number of database queries per second should drop significantly when cache hit rate goes up.

How Twitter/X Uses Caching

Twitter (now X) serves hundreds of millions of users with timelines that are assembled in milliseconds. Almost none of that is computed live.

- **Timeline pre-computation:** when you tweet, Twitter's fan-out service pushes your tweet ID into the timeline cache of all your followers immediately — so when they open the app, their feed is already assembled and waiting.
- **Flock (Twitter's cache):** Twitter built a custom distributed in-memory graph store to cache social graph relationships — who follows whom — because re-querying this from a database on every request would be impossible at scale.
- **Redis for rate limiting:** Twitter uses Redis counters to track API calls per user, incrementing a key per second and checking it against limits — all in memory, in microseconds.
- **Tweet objects in Memcached:** the raw tweet object (text, author, metadata) is cached separately from the timeline. When your feed loads, it assembles tweet IDs from one cache and tweet content from another.
- **Result:** a timeline request at Twitter may touch zero databases for the vast majority of active users.

How Netflix Uses Caching

Netflix streams to 260 million subscribers. Most of those bytes never come from Netflix's own origin servers — they come from caches placed inside ISPs.

- **Open Connect CDN:** Netflix's own private CDN. They install dedicated cache servers (called OCA — Open Connect Appliances) directly inside internet service providers worldwide. When you press play, the video comes from hardware sitting in your own ISP's building.
- **Pre-positioning popular content:** Netflix analyzes what shows are trending and proactively pushes those files to edge caches before the evening peak — not after the first user requests them.
- **EVCache (Netflix's custom cache):** built on top of Memcached, deployed across AWS availability zones. Caches user preferences, watch histories, search results, personalization data, and A/B test configurations.
- **Scale:** EVCache handles over 30 million requests per second with a hit rate above 99% — protecting backend microservices from direct traffic entirely.

How E-Commerce Uses Caching

E-commerce platforms face enormous cache challenges: product data must look fresh, inventory must be accurate, but millions of pages must load in under a second.

- **Product pages:** product name, description, images, and reviews are cached aggressively (long TTL) — they change rarely. Inventory count is kept separately with a very short TTL (seconds) or pulled live.
- **Search results:** search queries like "blue running shoes size 10" are cached with a medium TTL. The same query from the next user hits the cache — not the search index.
- **Cart & checkout:** never cached — always read live from the database, since stale cart data means charging users wrong prices.
- **Flash sales (Amazon, Flipkart):** before a sale launch, all sale product pages are pre-loaded into CDN and application caches. The huge traffic spike hits the cache, not the database.
- **Homepage personalisation:** cached per user segment (not per user) — e.g. "recommended for electronics buyers" is one cache key, not one per individual person.

Caching for API Rate Limiting

One of the most common non-obvious uses of caching is **rate limiting** — tracking how many requests a user has made in a time window and blocking them if they exceed the limit.

- **Why Redis?** rate limit counters need to be fast (sub-millisecond), atomic (no race conditions), and auto-expiring (reset after the time window). Redis's INCR command and EXPIRE perfectly solve all three.
- **How it works:** on each request, INCR a key like `rate:user:4521:minute:14:23`. If the counter exceeds the limit, reject the request. The key auto-expires after 60 seconds — resetting the counter.
- **Sliding window vs fixed window:** fixed window resets at the top of each minute. Sliding window tracks requests in a rolling 60-second period using Redis sorted sets — more accurate but more complex.
- **Used by:** GitHub API, Twitter API, Stripe API, and most cloud services — all use Redis-based rate limiting to protect their backends from abuse.

Mistakes Every Developer Makes with Caching

1

Caching too early

Adding caching before profiling to confirm where the actual bottleneck is. Cache the slowest queries first — not everything blindly.

2

Not logging cache misses

Without observability, you can't know if your hit rate is 5% or 95%. Always instrument cache hits, misses, and evictions with metrics.

3

Forgetting to test without the cache

If your application breaks when Redis is down, you have a dependency problem. The cache should degrade gracefully to a slow-but-working system.

4

Identical TTL for all keys

Not all data changes at the same rate. User profiles, product prices, and live stock levels should all have very different TTLs.

Key Terms at a Glance

Cache Hit / Miss

Hit = data found in cache, served instantly. Miss = data not in cache, must be fetched from source.

TTL (Time To Live)

The number of seconds a cached item stays valid before it automatically expires and must be re-fetched.

Eviction

Removing an item from cache to make room for new data, based on a policy like LRU or LFU.

Cache Stampede

A spike of database load when many requests simultaneously miss the cache for the same popular key.

Cache-Aside / Write-Through / Write-Back

The three main patterns for deciding when and how to populate and update the cache relative to the database.

Bloom Filter

A space-efficient data structure that tells you with certainty if a key does not exist — used to block cache penetration attacks.

Now You Understand Caching, Fully

From browser headers to Redis clusters, from cache stampedes to Netflix's ISP-level edge servers — caching is the single biggest lever you have for making systems fast at scale.



Like this post



Repost it



Follow for more